

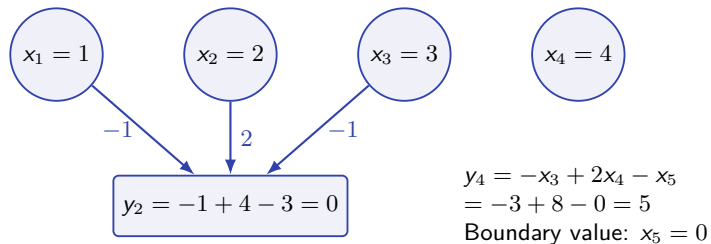
E09: A tridiagonal stencil is a matrix-vector product

$$T = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}, \quad T\mathbf{x} = \begin{bmatrix} 2 - 2 \\ -1 + 4 - 3 \\ -2 + 6 - 4 \\ -3 + 8 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 5 \end{bmatrix}.$$

Here $p = q = 1$ and $\text{nnz}(T) = 3n - 2 = 10$. For $y \leftarrow y + T\mathbf{x}$, skipping structural zeros gives 20 update flops rather than 32.

Try it: Why is the last entry nonzero although the interior entries vanish?

E09 figure: A local stencil produces one output



The tridiagonal matrix couples each sample only to its immediate neighbours.

Triangular Matrix Multiplication

For upper triangular A, B, C : $c_{ij} = c_{ij} + \sum_{k=i}^j a_{ik}b_{kj}$.

Algorithm 1.2.1

```
for i=1:n
  for j=i:n
    for k=i:j
      C(i,j)=C(i,j)+A(i,k)*B(k,j);
    end
  end
end
```

Flops: $\approx n^3/3$ (vs $2n^3$ for full).

E10: Restrict the summation range

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 3 & 2 \\ 0 & 0 & 1 \end{bmatrix}.$$

For c_{23} , only $k = 2, 3$ contribute:

$$c_{23} = u_{22}v_{23} + u_{23}v_{33} = 4 \cdot 2 + 5 \cdot 1 = 13.$$

$$UV = \begin{bmatrix} 2 & 7 & 7 \\ 0 & 12 & 13 \\ 0 & 0 & 6 \end{bmatrix}.$$

For $n = 3$, $\sum_{i \leq j} (j - i + 1) = 10$ updates: 20 flops rather than 54 flops for dense updates.

Band Storage

For $A \in \mathbb{R}^{m \times n}$ with lower/upper bandwidths p, q ,

$$AB(i - j + q + 1, j) = a_{ij}, \quad AB \in \mathbb{R}^{(p+q+1) \times n}.$$

For a 5×5 matrix with $p = 1, q = 2$, there are **four** storage rows:

$$AB = \begin{bmatrix} * & * & a_{13} & a_{24} & a_{35} \\ * & a_{12} & a_{23} & a_{34} & a_{45} \\ a_{11} & a_{22} & a_{33} & a_{44} & a_{55} \\ a_{21} & a_{32} & a_{43} & a_{54} & * \end{bmatrix}.$$

Stars mark unused positions. Banded gaxpy costs $2 \operatorname{nnz}(A) \leq 2n(p + q + 1)$ update flops.

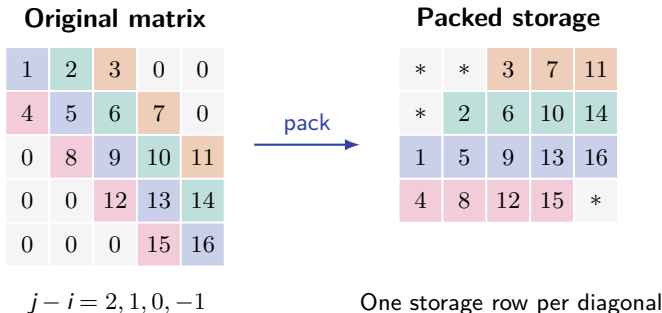
E11: Pack a band matrix explicitly

$$A = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 \\ 4 & 5 & 6 & 7 & 0 \\ 0 & 8 & 9 & 10 & 11 \\ 0 & 0 & 12 & 13 & 14 \\ 0 & 0 & 0 & 15 & 16 \end{bmatrix}, \quad AB = \begin{bmatrix} * & * & 3 & 7 & 11 \\ & 2 & 6 & 10 & 14 \\ 1 & 5 & 9 & 13 & 16 \\ 4 & 8 & 12 & 15 & * \end{bmatrix}.$$

With $p = 1$, $q = 2$, the storage row for $a_{24} = 7$ is $2 - 4 + 3 = 1$. For $x = 1$, $Ax = (6, 22, 38, 39, 31)^T$.

Try it: Where is $a_{43} = 12$ stored? How many actual matrix entries are retained? **Check:** $AB(4, 3) = 12$; 16 entries plus 4 unused storage positions.

E11 figure: Diagonals become storage rows



Trace the value 12: a_{43} maps to $AB(4, 3)$ because $4 - 3 + 2 + 1 = 4$.

Diagonal and Symmetric Matrices

- Diagonal: $D = \text{diag}(d_1, \dots, d_q)$. $Dx = d \cdot x$.
- Symmetric: $A^T = A$. Store only lower triangle using $A.\text{vec}$.
- Symmetric gaxpy (Algorithm 1.2.3) still $2n^2$ flops.

E12: Scaling and symmetry without dense storage

$$D = \text{diag}(2, -1, 3), \quad x = (1, 2, -1)^T \implies Dx = (2, -2, -3)^T.$$

For $S = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 4 \\ 0 & 4 & 5 \end{bmatrix}$ and $z = (1, 2, 3)^T$:

$$Sz = (4, 19, 23)^T.$$

Store the six lower-triangle positions instead of nine full-matrix positions. A stored off-diagonal entry s_{ij} contributes to *both* y_i and y_j : $y_i += s_{ij}z_j$, $y_j += s_{ij}z_i$.

Try it: Why does halving storage not automatically halve arithmetic?

Permutation Matrices

Rows of identity reordered. Represented by integer vector v : $P = I_n(v, :)$.

- $y = Px \Rightarrow y_i = x_{v_i}$.
- $P^T P = I$, $P^{-1} = P^T$.
- No flops but data movement.
- Famous: exchange $\mathcal{E}_n$, downshift $\mathcal{D}_n$, perfect shuffle $\mathcal{P}_{p,r}$.

E13: Reorder data and undo the permutation

$$v = (3, 1, 2), \quad P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad x = (10, 20, 30)^T.$$

$$Px = (30, 10, 20)^T, \quad P^T(Px) = (10, 20, 30)^T.$$

The inverse index vector is $(2, 3, 1)$, which is *not* the original vector. For any conformable A , PA reorders rows; AP^T reorders columns using v .

Try it: Does this operation require floating-point multiplication?

Block Matrices

A block matrix has submatrices as entries.

$$A = \begin{bmatrix} A_{11} & \cdots & A_{1r} \\ \vdots & \ddots & \vdots \\ A_{q1} & \cdots & A_{qr} \end{bmatrix}$$

Operations: scaling, transposition, addition, multiplication (if conformable).

Theorem (Theorem 1.3.1)

If $C = AB$ with conformable block partition, then $C_{\alpha\beta} = \sum_{\gamma} A_{\alpha\gamma} B_{\gamma\beta}$.

Row-blocked:

$$y_i = y_i + A_i x, \quad i = 1 : q.$$

Column-blocked:

$$y = y + \sum_{j=1}^r A_j x_j.$$

These break the operation into smaller gaxpys and can improve locality when the partition matches data reuse.

Block Matrix Multiplication

For conformable blocks, initialize $C = 0$ and accumulate

$$C_{ij} \leftarrow C_{ij} + A_{ik}B_{kj} \quad \text{for every } i, j, k.$$

For a two-by-two block partition,

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}.$$

At a fixed k , the **block outer-product update** is

$$C \leftarrow C + A_{:k}B_{k:},$$

where $A_{:k}$ is a block column and $B_{k:}$ a block row.

Each block multiplication can use a Level-3 BLAS routine.

E14: Compute a block product by hand

Let $I = I_2$ and

$$A = \begin{bmatrix} I & 2I \\ 0 & I \end{bmatrix}, \quad B = \begin{bmatrix} 3I & I \\ I & 0 \end{bmatrix}.$$

$$C_{11} = I(3I) + (2I)I = 5I, \quad C_{12} = I(I) + (2I)0 = I,$$

$$C_{21} = I, \quad C_{22} = 0, \quad AB = \begin{bmatrix} 5 & 0 & 1 & 0 \\ 0 & 5 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

Try it: If $x = (1, 2, 3, 4)^T$, compute Ax using two vector blocks. **Check:** $Ax = (7, 10, 3, 4)^T$.

Kronecker Product

Definition: $B \otimes C$ is block matrix with (i, j) block $b_{ij}C$. Properties:

$$\begin{aligned}(B \otimes C)^T &= B^T \otimes C^T, \\ (B \otimes C)(D \otimes F) &= BD \otimes CF, \\ (B \otimes C)^{-1} &= B^{-1} \otimes C^{-1}.\end{aligned}$$

The product identity requires conformable factors. The inverse identity requires square, nonsingular factors. Permutations connect $B \otimes C$ and $C \otimes B$.

E15: Expand a small Kronecker product

$$B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}.$$

$$B \otimes C = \begin{bmatrix} C & 2C \\ 0 & C \end{bmatrix} = \begin{bmatrix} 2 & 0 & 4 & 0 \\ 0 & 3 & 0 & 6 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}.$$

Try it: Where does the scalar $b_{12} = 2$ appear in the expanded matrix? It scales the entire upper-right block. The result is 4×4 , not 2×2 .

E15 figure: Each scalar expands into a block

$$\begin{array}{|c|c|} \hline B & \\ \hline 1 & 2 \\ \hline 0 & 1 \\ \hline \end{array} \otimes \begin{array}{|c|c|} \hline C & \\ \hline 2 & 0 \\ \hline 0 & 3 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline B \otimes C & \\ \hline 2 & 0 & 4 & 0 \\ \hline 0 & 3 & 0 & 6 \\ \hline 0 & 0 & 2 & 0 \\ \hline 0 & 0 & 0 & 3 \\ \hline \end{array}$$

The scalar $b_{12} = 2$ becomes the entire upper-right block $2C$.

Try it: Which block must vanish when $b_{21} = 0$?

Vec and Reshaping

$\text{vec}(X)$ stacks columns of X . Key identity:

$$Y = CXB^T \iff \text{vec}(Y) = (B \otimes C) \text{vec}(X).$$

For dense $n \times n$ factors, two matrix products cost $O(n^3)$; explicitly multiplying the $n^2 \times n^2$ Kronecker matrix by a vector costs $O(n^4)$.

E16: Verify the vec identity numerically

Use B, C from the previous example and $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.

$$CX = \begin{bmatrix} 2 & 4 \\ 9 & 12 \end{bmatrix}, \quad Y = CXB^T = \begin{bmatrix} 10 & 4 \\ 33 & 12 \end{bmatrix}.$$

$$\text{vec}(X) = \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix}, \quad (B \otimes C) \text{vec}(X) = \begin{bmatrix} 10 \\ 33 \\ 4 \\ 12 \end{bmatrix} = \text{vec}(Y).$$

Try it: What goes wrong if you stack rows instead of columns?

Hamiltonian and Symplectic

Hamiltonian:

$$M = \begin{bmatrix} A & F \\ G & -A^T \end{bmatrix}$$

with F, G symmetric, or $JMJ^T = -M^T$ with $J = \begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix}$.

Symplectic: $S^T JS = J$.

E17: A structure-preserving shear

For $J = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ and $S = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}$,

$$JS = \begin{bmatrix} 0 & 1 \\ -1 & -h \end{bmatrix}, \quad S^T JS = \begin{bmatrix} 1 & 0 \\ h & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ -1 & -h \end{bmatrix} = J.$$

Thus the shear is symplectic for any real h . For the Hamiltonian matrix $M = \begin{bmatrix} 0 & 1 \\ -4 & 0 \end{bmatrix}$, explicit Euler gives

$$E = I + hM, \quad \det E = 1 + 4h^2, \quad E^T J E = (1 + 4h^2)J.$$

Try it: Is explicit Euler symplectic when $h = 1/2$?

Strassen's Algorithm

For 2×2 block product:

- Conventional: 8 multiplications.
- Strassen: 7 multiplications (18 additions).
- Recursively applied yields complexity $O(n^{\log_2 7}) \approx O(n^{2.807})$.

Practical: use conventional for small blocks (threshold $n_{\min}$).

E18: Seven products, with actual numbers

Use $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

Product	Formula	Value
M_1	$(a + d)(e + h)$	65
M_2	$(c + d)e$	35
M_3	$a(f - h)$	-2
M_4	$d(g - e)$	8
M_5	$(a + b)h$	24
M_6	$(c - a)(e + f)$	22
M_7	$(b - d)(g + h)$	-30

E19: Recombine and derive the exponent

$$c_{11} = M_1 + M_4 - M_5 + M_7 = 65 + 8 - 24 - 30 = 19,$$

$$c_{12} = M_3 + M_5 = -2 + 24 = 22,$$

$$c_{21} = M_2 + M_4 = 35 + 8 = 43,$$

$$c_{22} = M_1 - M_2 + M_3 + M_6 = 65 - 35 - 2 + 22 = 50.$$

For matrix blocks, recursively repeat the same seven-product construction:

$$T(n) = 7T(n/2) + O(n^2) \implies T(n) = O(n^{\log_2 7}).$$

Try it: Why might ordinary multiplication still be faster for a 2×2 matrix? Extra additions, temporary storage, and overhead can dominate at small sizes.

Discrete Fourier Transform: $y = F_n x$, $[F_n]_{kj} = \omega_n^{(k-1)(j-1)}$, $\omega_n = e^{-2\pi i/n}$.

Radix-2 FFT (Algorithm 1.4.1): recursively split even/odd indices.

$$y(1 : m) = y_T + d \cdot y_B, \quad y(m+1 : n) = y_T - d \cdot y_B,$$

with $m = n/2$. Work: $O(n \log n)$ rather than $O(n^2)$. Constants depend on complex-arithmetic conventions.

E20: A four-point DFT

Use zero-based frequency indices $k = 0, 1, 2, 3$, $\omega_4 = -i$, and $x = (1, 2, 3, 4)^T$.

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}.$$

$$F_4 x = \begin{bmatrix} 10 \\ -2 + 2i \\ -2 \\ -2 - 2i \end{bmatrix}.$$

Try it: Which output is the sum of all entries? Which two outputs are conjugates? $y_0 = 10$; y_1 and y_3 are conjugates because the input is real.

E21: Trace the FFT butterflies

Split by **zero-based** even/odd indices:

$$\mathbf{x}_{\text{even}} = (1, 3)^T, \quad \mathbf{x}_{\text{odd}} = (2, 4)^T.$$

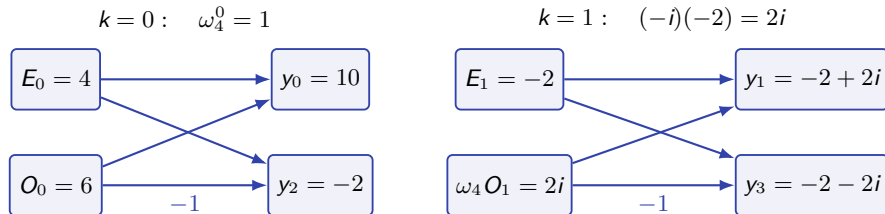
Two-point transforms: $E = (4, -2)^T$, $O = (6, -2)^T$.

k	E_k	$\omega_4^k O_k$	$y_k = E_k + \omega_4^k O_k$	$y_{k+2} = E_k - \omega_4^k O_k$
0	4	6	10	-2
1	-2	2i	$-2 + 2i$	$-2 - 2i$

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

Try it: Which MATLAB slices select the zero-based even and odd samples? **Check:** `x(1:2:end)` and `x(2:2:end)`.

E21 figure: Two FFT butterflies



At each right-hand node, sum the incoming contributions.

Upper output: $E_k + \omega_4^k O_k$; lower output: $E_k - \omega_4^k O_k$.

The two-point transforms of $(1, 3)$ and $(2, 4)$ supply E and O .

Fast Sine/Cosine Transforms

DST and DCT are related to DFT via structured vectors.

- DST: $y_k = \sum_{j=1}^{m-1} \sin(kj\pi/m)x_j$.
- DCT: $y_k = \frac{x_0}{2} + \sum_{j=1}^{m-1} \cos(kj\pi/m)x_j + \frac{(-1)^k x_m}{2}$.

Algorithms 1.4.2 and 1.4.3 compute them via FFT in $O(m \log m)$.

E22: A sine mode and a boundary condition

Take $m = 4$, $x = (1, 0, -1)^T$, and the unnormalized DST-I matrix

$$S = \begin{bmatrix} s & 1 & s \\ 1 & 0 & -1 \\ s & -1 & s \end{bmatrix}, \quad s = \frac{1}{\sqrt{2}}.$$

$$Sx = \begin{bmatrix} s - s \\ 1 + 1 \\ s - s \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}.$$

Only the second sine mode is present. For $T = \text{tridiag}(-1, 2, -1)$ of size 3,

$$Tx = 2x.$$

Try it: Why do sine modes suit homogeneous Dirichlet boundary conditions?

E23: A cosine transform from an even extension

Use the endpoint-half-weight DCT-I on $x = (1, 2, 1)$ with $m = 2$:

$$y_k = \frac{x_0}{2} + x_1 \cos(k\pi/2) + \frac{(-1)^k x_2}{2}.$$

$$y_0 = \frac{1}{2} + 2 + \frac{1}{2} = 3, \quad y_1 = \frac{1}{2} - \frac{1}{2} = 0, \quad y_2 = \frac{1}{2} - 2 + \frac{1}{2} = -1.$$

The even extension is $(1, 2, 1, 2)$, whose DFT is $(6, 0, -2, 0)$. Its first three entries are $2y$. FFTW's unnormalized DCT-I uses this doubled convention.

Try it: Why must you check normalization before comparing two transform libraries?

Haar Wavelet Transform

Convention: orthonormal Haar analysis, for $n = 2^q$. For each adjacent pair, compute an average and a detail:

$$a_j = \frac{x_{2j-1} + x_{2j}}{\sqrt{2}}, \quad d_j = \frac{x_{2j-1} - x_{2j}}{\sqrt{2}}.$$

Recursively transform the averages and append the details.

```
function y = haar_analysis(x)
    if numel(x)==1, y=x; return; end
    a=(x(1:2:end)+x(2:2:end))/sqrt(2);
    d=(x(1:2:end)-x(2:2:end))/sqrt(2);
    y=[haar_analysis(a); d]; % x is a column vector
end
```

Work is $O(n)$. This convention includes normalization and specifies coefficient order.

E24: Averages and details on four samples

For $x = (2, 4, 6, 8)^T$, compute adjacent-pair coefficients:

$$a^{(1)} = (3\sqrt{2}, 7\sqrt{2})^T, \quad d^{(1)} = (-\sqrt{2}, -\sqrt{2})^T.$$

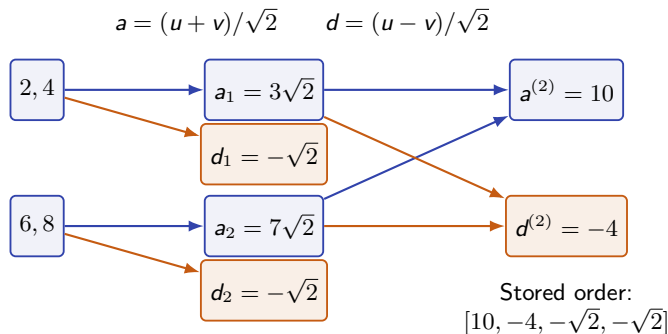
Transform the averages once more:

$$a^{(2)} = \frac{3\sqrt{2} + 7\sqrt{2}}{\sqrt{2}} = 10, \quad d^{(2)} = \frac{3\sqrt{2} - 7\sqrt{2}}{\sqrt{2}} = -4.$$

$$y = (a^{(2)}, d^{(2)}, d_1^{(1)}, d_2^{(1)})^T = (10, -4, -\sqrt{2}, -\sqrt{2})^T.$$

Try it: What would the detail coefficients be for a constant input?

E24 figure: The Haar decomposition tree



Blue nodes carry averages to the next level. Orange nodes retain details.

E25: Reconstruct and check energy

Invert each pair using $x_{2j-1} = (a_j + d_j)/\sqrt{2}$ and $x_{2j} = (a_j - d_j)/\sqrt{2}$.

$$a_1^{(1)} = (10 - 4)/\sqrt{2} = 3\sqrt{2}, \quad a_2^{(1)} = (10 + 4)/\sqrt{2} = 7\sqrt{2}.$$

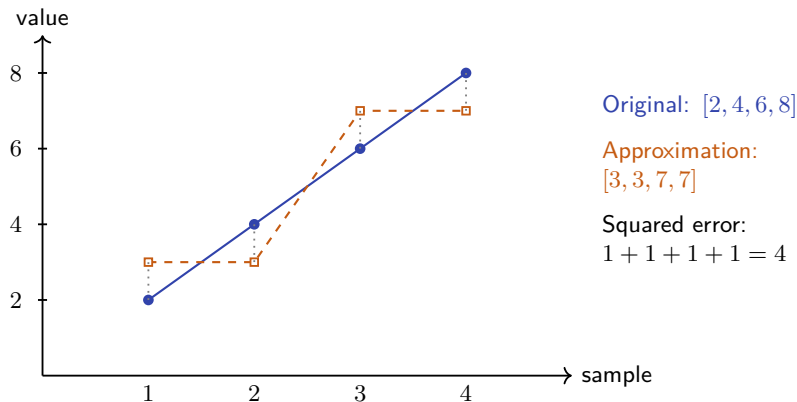
Then $x_1 = (3\sqrt{2} - \sqrt{2})/\sqrt{2} = 2$, $x_2 = 4$, $x_3 = 6$, $x_4 = 8$.

$$\|x\|_2^2 = 4 + 16 + 36 + 64 = 120 = 100 + 16 + 2 + 2 = \|y\|_2^2.$$

If the fine details are set to zero, $\tilde{x} = (3, 3, 7, 7)^T$ and $\|x - \tilde{x}\|_2^2 = 4$.

Try it: Where is that lost energy visible in the coefficients?

E25 figure: What disappears when fine details are removed?



$(-\sqrt{2})^2 + (-\sqrt{2})^2 = 4$: discarded coefficient energy equals reconstruction error energy.

Vector Processing and Pipelining

- Vector registers length v_L .
- Pipelining: once filled, one result per cycle.
- Data motion cost: loads/stores.
- Arithmetic-to-data ratio matters.

Example: vector addition $z = x + y$ requires $(\tau_{\text{add}}/v_L + 1)n$ arithmetic cycles and $3(\tau_{\text{data}}/v_L + 1)n$ data cycles.

Gaxpy vs Outer Product Data Motion

Assume $A \in \mathbb{R}^{n \times n}$ does not remain in fast memory, while the vectors do. Use 8-byte values.

Operation	Arithmetic	Ideal transferred bytes
$y \leftarrow y + Ax$	$2n^2$	$8(n^2 + 3n)$
$A \leftarrow A + xy^T$	$2n^2$	$8(2n^2 + 2n)$

The outer update reads *and writes* the matrix. Gaxpy reads the matrix.

For large n , matrix traffic differs by approximately a factor of two.

Actual transfers depend on cache behavior and the implementation.

Stride and Cache Blocking

- Unit stride accesses adjacent entries in memory.
- MATLAB uses column-major storage: entries in a column are contiguous.
- Blocking aims to reuse data within a fast-memory level.

A simple capacity model for three square blocks of width b is

$$3b^2 \leq M, \quad b \leq \sqrt{M/3},$$

where M counts **scalar values**, not bytes.

This is a feasibility estimate, not a universal optimal block size.

E26: Trace MATLAB memory addresses

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

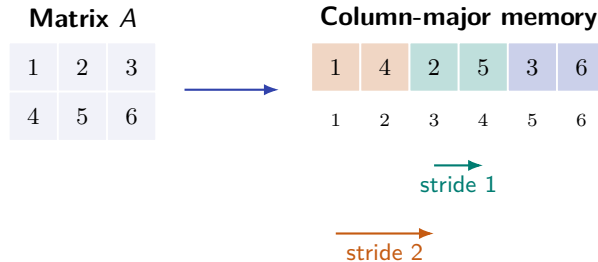
Column-major memory order: $[1, 4, 2, 5, 3, 6]$.

Access	One-based memory positions	Stride
$A(:, 2) = [2, 5]^T$	3, 4	1
$A(1, :) = [1, 2, 3]$	1, 3, 5	2

$$\text{position}(i, j) = i + (j - 1)m, \quad m = 2.$$

Try it: In the j, k, i matmul order, which arrays have contiguous inner-loop access? **Check:** $A(:, k)$ and $C(:, j)$; the scalar $B(k, j)$ is reused.

E26 figure: The same matrix has different access strides



$A(:, 2)$ uses positions 3, 4. $A(1, :)$ uses positions 1, 3, 5.

Position labels use MATLAB indexing. Adjacent entries of a column are adjacent in memory.

E27: A concrete cache budget

Assume 8-byte values and a 32 KiB fast-memory budget for three blocks.

$$24b^2 \leq 32768 \implies b \leq 36.95.$$

Width b	Three-block footprint	Within budget?
32	$24 \cdot 32^2 = 24576$ bytes = 24 KiB	Yes
64	$24 \cdot 64^2 = 98304$ bytes = 96 KiB	No

Try it: Why is $b = 32$ only a candidate, rather than a proven optimal size? Other data, cache conflicts, vector widths, and packing overhead also matter.

$C = AB$ with p processors. Block matrices:

$$C = (C_{ij})_{M \times N}, \quad A = (A_{ik})_{M \times R}, \quad B = (B_{kj})_{R \times N}.$$

Task (i, j) : $C_{ij} = C_{ij} + \sum_{k=1}^R A_{ik} B_{kj}$.

Task Distribution

- Block distribution: contiguous blocks.
- Block-cyclic distribution: cyclic assignment.

Block-cyclic assignment can improve balance when expensive blocks cluster; the benefit depends on the workload.

E28: Block versus cyclic assignment

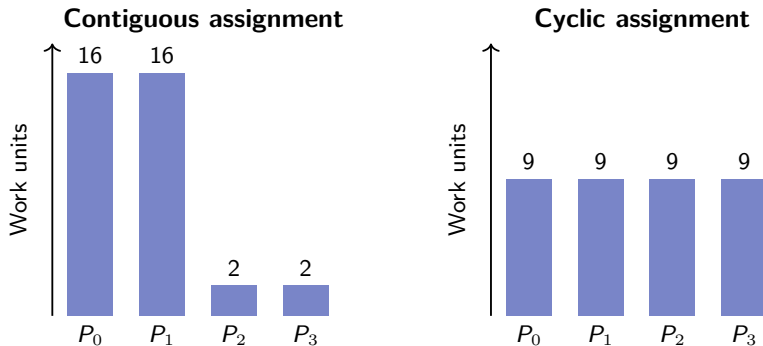
Eight tasks have illustrative costs $[8, 8, 8, 8, 1, 1, 1, 1]$ on four processors.

Assignment	P_0	P_1	P_2	P_3
Contiguous task pairs	$8 + 8$	$8 + 8$	$1 + 1$	$1 + 1$
Cyclic, block size 1	$8 + 1$	$8 + 1$	$8 + 1$	$8 + 1$

Ignoring communication, completion times are 16 and 9 units, respectively. Total work is 36, so the ideal balanced time is $36/4 = 9$.

Try it: Would cyclic assignment help if every task had exactly the same cost?

E28 figure: Load balance determines completion time



For the same 36 work units, ideal completion time falls from 16 to 9 when these tasks are assigned cyclically.

Communication Overhead

A simple message-time model is

$$T_{\text{message}} = \alpha + \beta\eta,$$

where α is latency, η is the number of words, and β is time per word. Assume balanced arithmetic with $T_{\text{arith}} = T_1/p$ and no overlap:

$$T_p = T_1/p + T_{\text{data}}, \quad S(p) = \frac{T_1}{T_p} = \frac{p}{1 + T_{\text{data}}/T_{\text{arith}}}.$$

Load imbalance and other overheads can further reduce speedup.

E29: Compute the speedup, including communication

Suppose $T_1 = 8$ s, $p = 4$, and $T_{\text{data}} = 0.5$ s, with balanced work and no overlap.

$$T_{\text{arith}} = 8/4 = 2 \text{ s}, \quad T_4 = 2 + 0.5 = 2.5 \text{ s}, \quad S(4) = 8/2.5 = 3.2.$$

Efficiency: $S(4)/4 = 0.8 = 80\%$. For $\alpha = 10 \mu\text{s}$, $\beta = 0.01 \mu\text{s}/\text{word}$, $\eta = 1000$ words:

$$T_{\text{message}} = 10 + 0.01(1000) = 20 \mu\text{s}.$$

Try it: How long do 100 separate 10-word messages take in the same model? **Check:** $100(10 + 0.1) = 1010 \mu\text{s}$, ignoring overlap.

Shared Memory Methods

- Method 1: load one block at a time; high redundancy.
- Method 2: pre-load all needed A and B blocks; better if local memory large.

Trade-off: local memory capacity vs communication cost.

Barrier Synchronization

```
Initialize C, A, B  
barrier  
Compute  $C = C + AB$ 
```

Barrier ensures all processors finish initialization before computation.

Each processor has local memory and exchanges messages with its neighbors. **Cannon's algorithm on**

a $q \times q$ processor torus:

- 1 Initially processor (i, j) owns A_{ij} and B_{ij} , with zero-based indices.
- 2 Skew A left by i positions in row i and B up by j positions in column j .
- 3 Multiply local blocks and accumulate the local output.
- 4 Shift A left once and B up once, then repeat for q products total.

The initial alignment ensures that matching inner indices meet on each processor.

E30: Cannon's algorithm on four processors

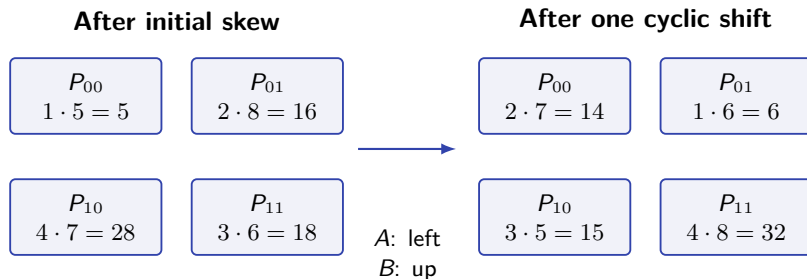
Use scalar blocks of $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. After the initial skew, processor (i, j) starts with $k = (i + j) \bmod 2$.

Processor	First local product	After one shift	Sum
P_{00}	$a_{00}b_{00} = 5$	$a_{01}b_{10} = 14$	19
P_{01}	$a_{01}b_{11} = 16$	$a_{00}b_{01} = 6$	22
P_{10}	$a_{11}b_{10} = 28$	$a_{10}b_{00} = 15$	43
P_{11}	$a_{10}b_{01} = 18$	$a_{11}b_{11} = 32$	50

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

Try it: Why must the two local blocks have the same inner index k ?

E30 figure: Two rounds on a processor grid



Accumulate at each processor: $C = \begin{bmatrix} 5 + 14 & 16 + 6 \\ 28 + 15 & 18 + 32 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$

Rows and columns wrap around on the processor torus. Each processor accumulates its own entry of C .

Summary

- Matrix multiplication has multiple equivalent formulations.
- Structure reduces storage and flops.
- Block algorithms enable high-performance (Level-3 BLAS).
- Fast transforms (FFT, Haar) use divide-and-conquer.
- Vectorization, caching, and parallelism are critical for large n .

- ① Gene H. Golub and Charles F. Van Loan.
Matrix Computations, 4th ed.
Johns Hopkins University Press, 2013.
- ② Roger A. Horn and Charles R. Johnson.
Matrix Analysis, 2nd ed.
Cambridge University Press, 2012.
- ③ Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong.
Mathematics for Machine Learning.
Cambridge University Press, 2020.
- ④ 王磊. 《矩阵理论与计算》.
Chinese-language course reference.

Key References

- BLAS: Lawson et al. (1979), Dongarra et al. (1988, 1990).
- Fast Transforms: Van Loan (FFT), Frigo & Johnson (2005).
- Parallel: Cannon (1969), ScaLAPACK.
- Strassen (1969), Ballard et al. (2011) for communication bounds.

Thank You